Claude Code for Biomolecular Simulation & Computational Chemistry

From First Principles to the BioKT Lab Configuration

David De Sancho

June 27, 2026

University of the Basque Country (UPV/EHU) & DIPC

Prelude

Live Demo

Prelude: `/dft-homo-lumo` on Guanidinium

What just happened

- One prompt: molecule name and charge
- Claude read a skill file encoding the full protocol
- Generated 3D geometry → DFT optimisation → SCF
- Extracted HOMO/LUMO energies and cube files
- Notebook visualised the frontier orbitals

Key numbers

	Guan. (+1)	Guan. (0)
HOMO (eV)	−15.08	−8.15
LUMO (eV)	−3.51	+1.47
Gap (eV)	11.57	9.63

Method: CAM-B3LYP / def2-TZVP

A Qualitative Shift in Research Practice

The jagged frontier

AI is superhuman at some tasks and surprisingly poor at others. Knowing which is which requires genuine domain expertise — not prompting skill.

The bottleneck shifts

AI agents can work on many tasks in parallel. The limiting factor is no longer execution — it is the quality of human judgment directing the work.

Expertise is rewarded, not replaced

The productivity gains from AI go disproportionately to those who already know their field deeply — because they know when to trust the output and when to push back.

The tool does not determine the outcome.

The expertise behind it does.

Outline

Part I — General Concepts

- What Claude Code is (and is not)
- Installation, interfaces, operating modes
- `CLAUDE.md`, skills & agents, memory, sessions
- Cost, privacy, and honest caveats

Part II — The BioKT Lab

- Scientific standards and compute infrastructure
- Skills library: QM, MD, enhanced sampling
- Daily workflow in practice

Closing

- What you can do starting Monday
- A caution on outsourcing judgment

Questions welcome throughout — or at the end.

Part I

General Concepts

What Is Claude Code?

What it IS

- An agentic AI coding assistant
- Lives in your terminal or IDE
- Reads and writes files, runs shell commands
- Calls external tools and browses the web
- Operates in a loop: plan → act → observe & validate
- Controlled by you at each step (or autonomously)

What it is NOT

- A chatbot (it *does* things, not just talks)
- An autocomplete plugin
- A one-shot code generator
- A replacement for understanding your domain

Key insight

You describe *intent*; Claude figures out the steps, executes them, and reports back.

Installation & Setup

Terminal (Mac / Linux)

- Install: `curl -fsSL https://claude.ai/install.sh | bash`
- Launch & auth: `claude` — browser opens for login
- Subscription or API key required
- Auto-updates in the background

macOS alternative: `brew install --cask claude-code`

VS Code Extension

- Extensions panel (`Ctrl+Shift+X`)
- Search **Claude Code** — publisher: Anthropic
- Provides: file tree, real-time diffs, inline edit review

First steps after install

- Edit `~/.claude/CLAUDE.md` — your standing brief
- Run `/init` inside a project to auto-generate a local `CLAUDE.md`

Cost & Privacy

Cost

- **Claude Pro** — ~$20/month; sufficient for most individual use
- **Claude Max** — higher usage limits; better for heavy or autonomous workflows
- **API / Console** — pay per token; useful for scripting and batch tasks
- No per-seat cost for the `claude` CLI itself

Privacy

- Anthropic does **not** use your conversations to train models by default
- Treat unpublished data with caution — avoid pasting novel sequences, unreleased structures, or confidential results into the session
- Work with file paths and structure; minimise raw scientific data in the conversation window

Rule of thumb

Treat Claude as an unaffiliated third-party service: fine for published data, cautious for unpublished results, avoid for anything under IP protection or confidentiality obligations.

Two Ways to Run Claude Code

Terminal

`$ claude`

- Text-only interface
- Fast and lightweight
- Steeper initial barrier
- Ideal for scripting and quick tasks

IDE Extension (VS Code)

Claude Code extension

- File tree and real-time diffs
- Review edits before accepting
- Lower barrier for new users
- Ideal for extended coding sessions
- Comfortable for remote work via SSH on different machines

Four Operating Modes

Plan

- Read-only exploration
- No files are changed
- Think before acting

Default

- Asks before every action
- Safest starting point
- Full control

Accept Edits

- File edits auto-applied
- Shell commands still confirmed
- Good middle ground

Auto

- Fully autonomous
- Runs commands freely
- Fastest; use with care

`Shift+Tab` cycles between modes | Start cautious, gain speed through trust

CLAUDE.md — Your Instruction File

What it is

- A Markdown file loaded into *every* session
- The “system prompt you write”
- Global: `~/.claude/CLAUDE.md`
- Local: `<project>/CLAUDE.md` (project-specific)
- Changes Claude’s behaviour without touching the model

Each session begins with a fresh context window — `CLAUDE.md` is how you carry knowledge across sessions.

What to put in it

- **Role & identity** — who you are, your domain
- **Standards** — correctness rules, safety constraints
- **Work preferences** — how to communicate, what to ask
- **Infrastructure** — machines, paths, software stack
- **Skill catalogue** — which `/skills` exist and when to use them
- **Project structure** — layout, git conventions

Skills & Slash Commands

What they are

- Reusable workflows stored as `SKILL.md` files
- Triggered by typing `/skill-name` in the prompt
- Claude reads the instructions and executes them
- Loaded at startup — restart after adding new skills

Scope

- **Global:** `~/.claude/skills/` — everywhere
- **Local:** `.claude/skills/` — project only; in git

The BioKT library lives in the shared repo `~/Software/Claude/Skills-BIOKT`, symlinked into `~/.claude/skills/`.

Not all `/commands` are skills — `/clear`, `/compact`, `/help`

Example

```
# In the Claude prompt:
/review-plan

# Claude stress-tests your plan:
#  - Identifies blind spots
#  - Checks best practices
#  - Flags wishful thinking
#  - Proposes alternatives
```

Why they matter

- Encode expertise *once*, reuse forever
- Share across machines and team members
- Build a personalised AI toolkit

Writing a Skill: Template Syntax

```
# ~/.claude/skills/my-skill.md

# Skill Name

One or two sentences describing what
this skill does and when to trigger it.

## Arguments

$ARGUMENTS can include:
- molecule name, charge, basis set

## Instructions

### Step 1: Read context
Check the current directory for
relevant input files.

### Step 2: Execute
Run the protocol, validating each step.
```

Key elements

- **No YAML frontmatter** — plain Markdown
- **Description** — 1–2 sentences at the top; Claude reads this to decide when to invoke it
- `$ARGUMENTS` — whatever the user types after `/skill-name`
- **Step sections** — each `### Step N` is a concrete action with clear output

Start simple; add steps iteratively as you test. One working step beats five broken ones.

Skills vs. Agents

Skills — `~/.claude/skills/`

Run *inside* the main conversation.

- Full chat history available
- Interactive; back-and-forth with user
- Triggered by `/skill-name`

Use when: user input needed mid-task, or conversation context matters.

Agents — `~/.claude/agents/`

Run in an *isolated* context window.

- No prior conversation; returns single result
- Autonomous; own model and tool list
- Can run in parallel

Use when: independent evaluation, parallel workloads, or focused bounded tasks.

Example: `/bash-parallelize` spawns N parallel agents for N independent replicas or mutants.

Writing an Agent: Template Syntax

```
# ~/.claude/agents/check-protocol.md

---
name: check-protocol
description: >
  Review simulation setup files for
  physical consistency and completeness
model: sonnet
tools:
  - Read
  - Glob
  - Grep
---

You are an independent simulation
reviewer. Check the mdp files, topology
    ,
and SLURM script in the project for:
- Thermostat/barostat mismatches
- Cutoff values outside best practice
```

YAML frontmatter

- **name** — unique identifier
- **description** — tells Claude when to invoke this agent
- **model** — `sonnet` / `opus` / `haiku`
- **tools** — restrict to what is needed; read-only for review tasks

Body (plain text)

- Role description
- Specific checks or steps
- Output format
- Explicit scope limits

Memory System

What is memory?

Each Claude session starts from scratch — it remembers nothing from previous conversations. The **memory system** solves this by writing persistent Markdown files that are loaded into every new session, carrying knowledge across conversations and machines.

Four memory types

Type	Stores
`user`	Role, expertise, preferences
`feedback`	Corrections & confirmed approaches
`project`	Goals, deadlines, decisions
`reference`	Where to find external resources

How it works

- Files in `~/.claude/projects/*/memory/`
- `MEMORY.md` index loaded in every session
- Claude saves memories proactively and on request
- Prevents repeating the same corrections
- Cross-session and cross-machine continuity

Session Management

What is context?

Context is everything Claude can currently "see": the conversation history, file contents read, tool outputs, and loaded instructions (`CLAUDE.md`, memory). The model can only act on what is in context — nothing else exists for it.

Why it matters

- Sessions grow; model attention dilutes
- Longer session $\neq$ better outcome
- Each turn becomes more expensive over time
- Strategic breaks produce cleaner results

Key principle

Compact at task boundaries — not in a crisis.

Core commands

- `/compact <hint>` — summarise & continue
  Hint tells Claude what to keep; be specific
- `/clear` — empty conversation, fresh start
  Best for switching to an unrelated task
- `/rewind` (`Esc Esc`) — restore a prior state
  Keep file edits but drop the discussion, or vice versa

Settings & Hooks

`settings.json` **key options**

model — which Claude model to use (`sonnet`, `opus`, `haiku`)

permissions.allow — commands that run without a confirmation prompt

hooks — shell prompts triggered on events (e.g. `PreCompact`, `PostToolCall`)

statusLine — dynamic terminal status display

MCP servers — connect Claude to external tools databases, APIs, local services; worth exploring once comfortable with the basics

Global: `~/.claude/settings.json`

Local: `settings.local.json`

Hook example

```
"hooks": {
  "PreCompact": [{
    "type": "prompt",
    "prompt": "Run /checkpoint
        save
    to checkpoint the session
    before compaction."
  }]
}
```

Before context is compressed, Claude automatically saves the session state.

Honest Caveats

What Claude does well

- Routine, well-documented protocols
- Boilerplate code, file I/O, scripting
- Aggregating and restructuring information
- Catching obvious errors in structured output

Where it falls short

- Domain-specific edge cases with sparse training data
- Subtle physical reasoning (e.g. rare force field combinations, unusual collective variables)
- Knowledge of *your* system, your history, your failures
- It can be confidently wrong

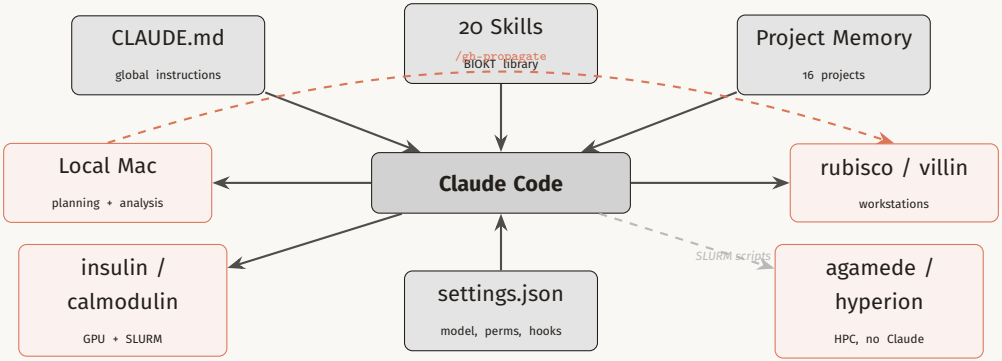
The bottom line

Claude produces drafts. Validation is always your responsibility.

Part II

The BioKT Lab Configuration

BioKT Setup at a Glance



CLAUDE.md: Scientific Standards

“Correctness over speed”

Simulation work goes on the scientific record — a mistake found three steps later costs more than pausing to ask.

1. **Ask rather than improvise**
   Ambiguous parameter? Force field? Stop and ask.
2. **Validate before proceeding**
   Check energy, *T*, *P*, RMSD after each stage.
3. **Never assume a prior step succeeded**
   Check exit codes, log files, output files explicitly.
4. **Prefer known-good protocols**
   No new flags without explanation and confirmation.
5. **Surface uncertainty explicitly**
   Protonation state? Box size? Ion conc.? Say so.
6. **Errors and warnings are blockers**
   Understand and resolve — never suppress.

Compute Infrastructure

Host	Role	SLURM	GPU	Claude
local (Mac)	Planning, analysis, writing	—	—	✓
rubisco	Remote workstation	—	yes	✓
villin	Remote workstation	—	yes	✓
insulin	GPU server	yes	yes	✓
calmodulin	GPU server (HP)	yes	yes	✓
agamede	EHU HPC cluster	yes	yes	×
hyperion	DIPC HPC cluster	yes	yes	×

`/gh-propagate` syncs `~/.claude` and `~/Software/Claude/Skills-BIOKT` to all ✓ machines via git

HPC clusters receive SLURM scripts prepared on interactive machines

Standard Project Layout

```
<ProjectName>/
|-- data/            # MD output (NOT
    git)
|   |-- *.xtc  *.gro  *.edr
|   `-- *.cpt  *.tpr  *.log
|-- analysis/
|   |-- notebooks/  # Jupyter (git)
|   `-- scripts/    # Python  (git)
|-- setup/           # mdp, top, itp,
    gro
|-- scripts/         # SLURM scripts
|-- README.md
`-- .gitignore
```

Key features

- `data/` excluded from git — large binary files
- Analysis code tracked for reproducibility
- Same layout on *all* machines — predictable paths
- `new_project.sh` scaffolds the tree and makes an initial commit

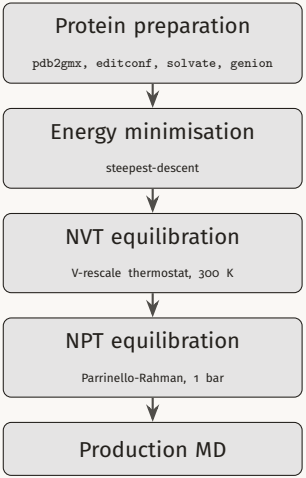
```
# Create a new project:
~/Research/Projects/new_project.sh
    \
    MyProjectName
```

Skills Library: BioKT Collection

Category	Skill	What it does
Simulation prep	`/gmx-prep`	GROMACS: EM → NVT → NPT → production
	`/gmx-install`	Build GROMACS + PLUMED from source
Enhanced sampling	`/plumed-us`	Umbrella sampling + WHAM
	`/plumed-metad`	Well-tempered metadynamics + sum_hills
	`/plumed-remd`	US-REMD, bias-exchange, PT-METAD
Parameterization	`/amber-parameterize`	SMILES → GAFF2 → GROMACS (.itp)
Quantum chem.	`/dft-homo-lumo`	PySCF DFT → HOMO/LUMO energies
Infrastructure	`/jobs`	Track simulation jobs in JOBS.md
	`/diy`	Execute proposed commands / actions
	`/sim-paths`	SSH pre-flight file check
	`/bash-parallelize`	Parallel subagents for independent loops
Knowledge mgmt	`/overview`	Aggregate TODO/CHECKPOINT across machines
	`/checkpoint`	Save/restore session context
	`/bitacora`	Append to lab notebook
	`/tyler`	PDF papers → markdown wiki

/gmx-prep: GROMACS MD Preparation

Workflow stages



Default settings

- Force field: `ff99SBws-STQ`
- Water model: `TIP4P/2005`
- Salt: 150 mM NaCl
- $T = 300$ K, $P = 1$ bar
- Timestep: 2 fs; cutoffs: 1.0 nm
- Electrostatics: PME

Advanced options

- AWH (free energy along a CV)
- H-REMD (Hamiltonian replica exchange)
- SLURM submission templates included
- Ligand integration via ACPYPE

Enhanced Sampling with PLUMED

`/plumed-us`

Umbrella Sampling

- Harmonic restraint windows
- Serial or MPI/REMD execution
- WHAM free energy profile
- Bootstrap error estimation
- Convergence check (7 stages)

`/plumed-metad`

Well-Tempered MetaD

- Adaptive Gaussian hills
- Biasfactor guidance
- `sum_hills` FES
- Trajectory reweighting
- Block error analysis

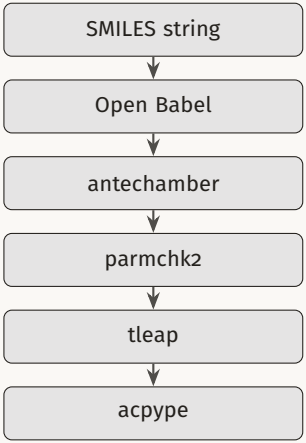
`/plumed-remd`

Replica Exchange

- US-REMD, bias-exchange
- PT-METAD temp. ladders
- `gmx_mpi -multidir`
- Trajectory demultiplexing
- WHAM with weights

All three skills produce validated `plumed.dat` files, mdp modifications, and SLURM scripts.

/amber-parameterize: Molecule Parameterization



Key choices

Force field: **GAFF2** (General AMBER FF)

Charges: **AM1-BCC** (default) or RESP

Python wrapper automates the full pipeline

Output & integration

Output: `LIG.itp`, `LIG.gro` (GROMACS-ready)

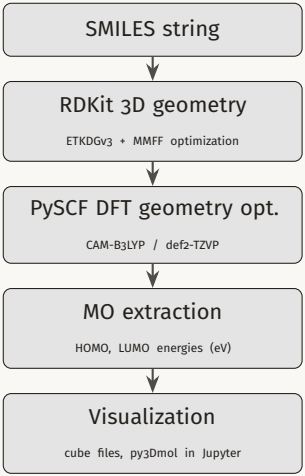
Integrates directly with `/gmx-prep`

DFT-optimized geometries supported as input

RDKit or Open Babel for initial 3D generation

/dft-homo-lumo: Quantum Chemistry

Pipeline



Settings

Functional: `CAM-B3LYP`

alts: B3LYP, wB97X-D3, PBE0

Basis set: `def2-TZVP`

high-acc: `def2-QZVP`

Convergence: 10^{-6} Hartree

Grid: level 4

Integration

DFT-optimized geometry → `/amber-parameterize`

RESP charges from DFT wavefunction

Useful for metal-binding residues and cofactors

Workflow Infrastructure

`/jobs`

Maintains `JOBS.md` — a table of all running and completed simulations across every machine.

- Job name, machine, SLURM ID
- Start date, status, notes
- Archive completed jobs

`/sim-paths`

SSH pre-flight check before job submission.

- Verifies all required files exist on the target machine
- Checks .gro, .top, .mdp, .dat, .sh
- Catches missing files before queue wait

`/bash-parallelize`

Spawns parallel subagents for independent tasks.

- Detects independence in bash loops
- Load-aware concurrency control
- Collects and reports results

Example: analyse 8 replicas in parallel instead of sequentially with a single `/bash-parallelize` call.

Knowledge & Session Management

`/overview`

Aggregates `CHECKPOINT.md` and `TODO.md` from all interactive machines via SSH. Produces a prioritised action plan for the day or week. Run from the local Mac.

`/checkpoint save` / `load`

Checkpoints the current session state to `CHECKPOINT.md`. Run before `/clear` to avoid losing context. Restored automatically at next session start if the file is present in the project directory.

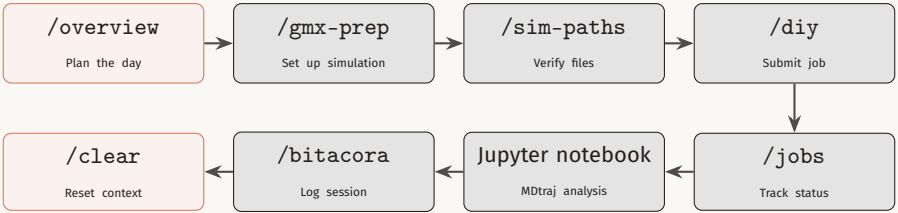
`/bitacora`

Appends a structured entry to `LABNOTEBOOK.md`: summary, decisions made, files touched, figures, next steps. Run before `/clear` to capture the session's scientific record.

`/tyler`

Converts a folder of PDF papers into a two-tier markdown wiki: an `index.md` (∼400 tokens/paper) and per-paper `papers/*.md` files read on demand. Keeps literature accessible within Claude's context window.

A Day in the Lab



Morning: `/overview` reveals what is running and what needs attention

Setup: Claude writes mdp files, prepares topology, generates SLURM script

Submit: `/diy` executes proposed commands; `/jobs` records the new entry

Analyse: notebooks with MDtraj, numpy, pandas — Claude helps debug and extend

End of session: `/bitacora` captures decisions; `/checkpoint save`; then `/clear`

What You Can Do Starting Monday

First steps

- Install: `curl -fsSL https://claude.ai/install.sh | bash`
- Write your `CLAUDE.md` — role, standards, infrastructure
- Run `/init` in one project to get a local `CLAUDE.md`
- Try `/dft-homo-lumo` on a molecule you know well

Build from there

- Use `/gmx-prep` for your next simulation setup
- Run `/overview` each morning to track active work
- End sessions with `/bitacora` and `/checkpoint save`
- Add skills as your workflows mature

Start with tasks you already know how to do. That way you can tell when Claude gets it right — and when it does not.

A Caution Before You Start

The real risk

- Not that Claude will fail spectacularly
- But that it will *succeed just enough* to prevent you from developing your own understanding
- A researcher who cannot evaluate the output is not doing research — they are transcribing an AI
- Deep intuitions about your system take years to build; they cannot be borrowed

Mollick calls this **cognitive surrender**: delegating your thinking so completely that you can no longer evaluate the output — even when it is wrong.

The rule

If you cannot tell whether Claude's output is right, you are not yet in a position to trust it.

Use Claude to go *faster* once you know where you are going.

Do not use it to avoid learning the map.

Further Reading & Resources

Documentation

- **Anthropic Claude Code docs**
  `https://code.claude.com/docs`
- **Claude Blattman**
  `https://claudeblattman.com`

Broader perspective

- **One Useful Thing** — Ethan Mollick
  `https://www.oneusefulthing.org`
- **Tyler Cowen** — AI and expertise
  `https://www.youtube.com/watch?v=aJlg6oOA_Js`

YouTube Tutorials

- John Kim
- Nick Saraev
- . . . *and many others*